



Hi-Z 오클루전 컬링 결과를 활용한 가시성 기반 애니메이션 갱신 기법

A Visibility-based Animation Update Method Using Hi-Z Occlusion Culling

장명은, 이상훈, 이종진, 정내훈 | 한국공학대학교 게임공학과

wkdauddns9@gmail.com, asm1133@naver.com, lij3640@gmail.com, nhjung@tukorea.ac.kr

개요

현대의 실시간 렌더링 분야에서는 Hi-Z 기반 오클루전 컬링과 같은 GPU 중심의 가시성 판별 기법이 널리 활용되고 있다. 그러나 이러한 가시성 정보는 주로 렌더링 단계 최적화에만 집중적으로 활용되며, 복잡한 제어로 인해 CPU에서 수행되어야 하는 애니메이션 갱신 비용 절감에는 충분히 활용되지 않고 있다. 본 연구는 GPU에서 판정한 가시성 정보를 활용하여, 보이지 않는 객체의 애니메이션 업데이트를 선택적으로 생략하는 기법을 제안한다. 제안 기법은 GPU에서 계산된 가시성 정보를 이전 프레임 기반으로 비동기적으로 활용하여, CPU-GPU 간 동기화 병목 없이 CPU 애니메이션 시스템에 반영할 수 있다. 이를 통해 역기구학이나 래그돌 물리 등 CPU 기반 처리가 요구되어 병목이 발생하는 애니메이션 환경에서, 애니메이션 연산량을 전체 객체 수가 아닌 실제 가시 객체 수에 비례하도록 감소시켜 성능을 향상한다.

연구 목적

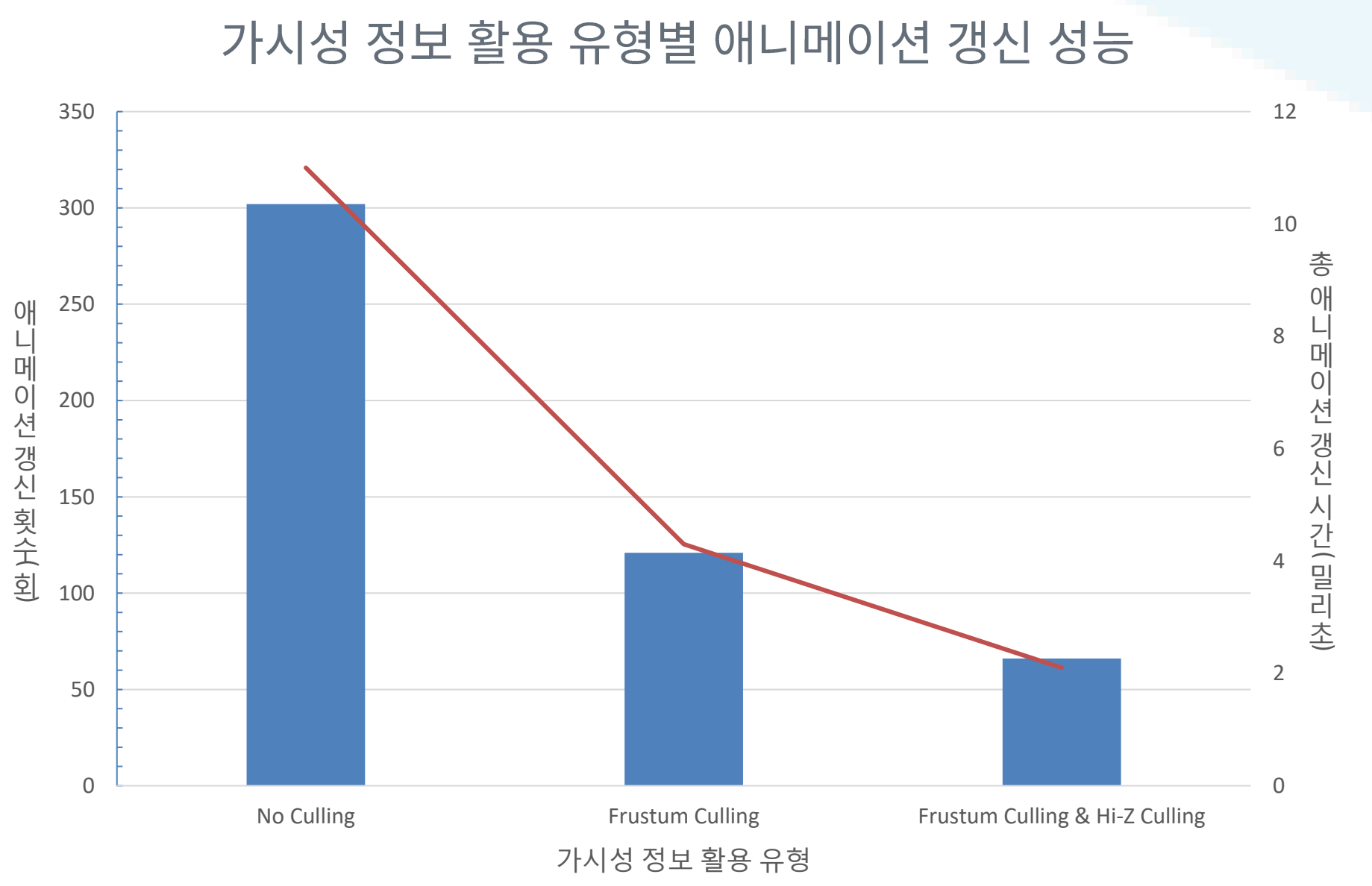
- 다수의 객체가 포함된 장면에서 보다 많은 객체를 가시성 기반으로 컬링하여 애니메이션 연산량을 절감한다.
- 래그돌, IK 등 복잡한 물리 연산이 애니메이션과 결합되어 CPU 중심 연산이 요구되는 환경에서 GPU가 생산한 가시성 데이터를 효율적으로 통합하는 방법을 제시한다.



[그림 1] 래그돌 물리가 적용되고 있는 장면

결과

애니메이션 컬링 적용 방법 중 컬링을 하지 않은 경우, 시야 절두체 컬링 정보만 활용한 경우, 시야 절두체 컬링 및 Hi-Z 오클루전 컬링 정보를 모두 활용한 경우를 비교군으로 설정하였다. 총 302개의 드로우 콜 대상 인스턴스에 대해 샘플 장면을 렌더링해, 총 3600회의 프레임 업데이트에 대한 평균 애니메이션 갱신 횟수, 평균 총 애니메이션 갱신 시간을 측정했다. 그 결과는 다음과 같다.

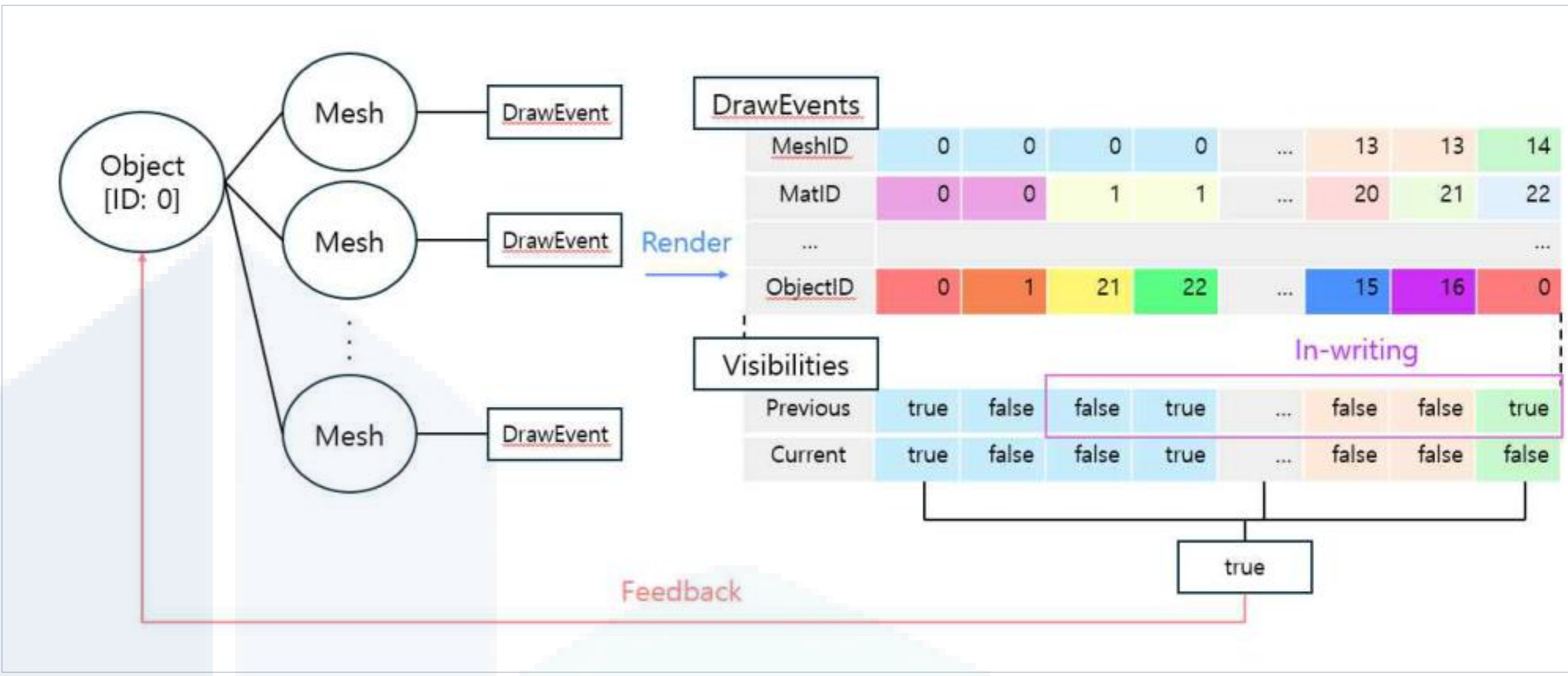


[그림 4] 가시성 정보 활용 유형별 애니메이션 갱신 성능 그래프

해석: readback과 객체 가시성 병합 비용이 매우 작아, CPU 기반 애니메이션이 주요 병목이 되는 장면에서 효과가 크다. 렌더링 품질을 유지하면서 보이지 않는 객체의 애니메이션 계산을 줄이는 것이 핵심이다.

성능 평가는 Ryzen 7 7800X3D, RTX 4070 Ti SUPER, Windows 11 환경에서 수행되었다.

가시성 기반 애니메이션 갱신



[그림 2] 시스템 개요

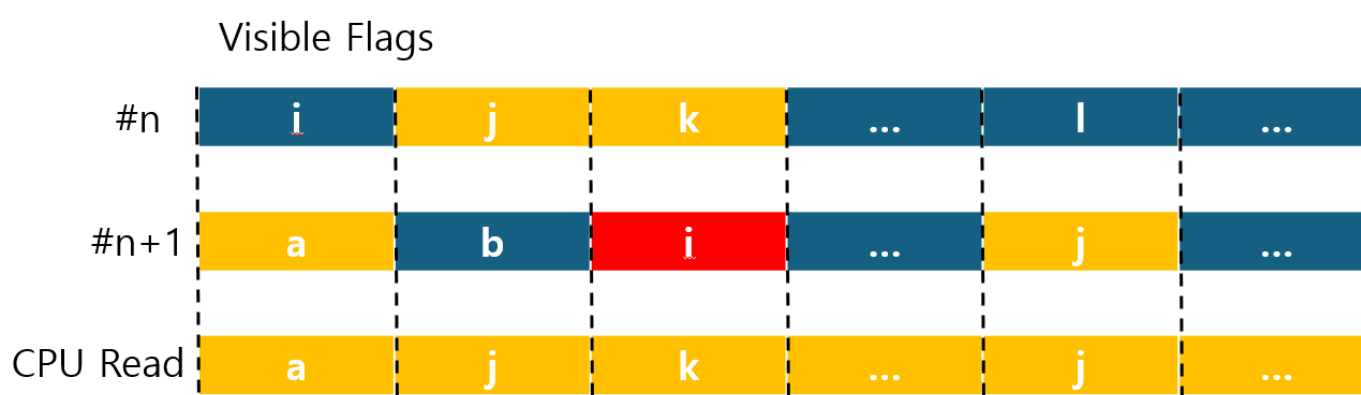
각 객체는 복수의 메시와 재질의 조합으로 구성되며, 렌더링 시 각 메시-재질 쌍에 대해 하나의 그리기 이벤트가 생성된다. 이때, 시야 절두체 컬링을 통해 비가시로 판정된 객체는 그리기 이벤트를 생성하지 않는다. 생성된 그리기 이벤트에는 인스턴스의 변환 행렬, 메시 및 재질 정보, 객체 식별자가 포함된다. 제출된 그리기 이벤트들은 인스턴싱을 위해 정렬되며, 렌더링 단계에서 Hi-Z 오클루전 컬링이 수행돼 각 이벤트에 대응하는 32비트 정수형 가시성 플래그가 생성된다. 이때 가시성은 1비트로 표현 가능하므로, 나머지 상위 비트에는 객체 식별자를 비트 단위 OR 연산을 통해 함께 저장한다. 이후 CPU에서 인스턴스 단위의 가시성 플래그들을 검사하며 객체 단위의 가시성을 결정하게 된다. 객체 단위의 가시성은 매번 비가시 판정으로 초기화된 뒤 객체를 구성하는 인스턴스가 하나라도 보이는 것으로 판별되면 가시 판정으로 갱신된다. CPU는 GPU와의 명시적인 동기화 없이 가시성 플래그 버퍼를 읽어들이며, 가시성 비트가 설정된 경우 상위 비트에 저장된 객체 식별자를 통해 해당 객체를 가시 상태로 판정한다. 이렇게 수집된 객체 단위의 가시성 정보는 애니메이션 시스템으로 전달된다. 애니메이션 시스템은 현재 시야에 포함된 객체들만을 대상으로 거리 및 마지막 갱신 시점을 고려한 스케줄링을 수행하며, 주어진 시간 예산 내에서 최대한 많은 애니메이션을 갱신한다. 이를 통해 컬링 결과와 LOD가 자연스럽게 반영된 애니메이션 최적화가 이루어진다.

GPU와 CPU 간의 명시적인 동기화를 제거할 경우, CPU는 GPU가 아직 쓰기 작업을 완료하지 않은 가시성 버퍼를 읽게 될 수 있다. 이로 인해 특정 객체의 가시성은 해당 객체를 구성하는 인스턴스들의 이전 프레임과 현재 프레임의 가시성 정보가 혼합된 상태로 판정될 수 있다. 각 인스턴스의 이전 프레임 가시성, 현재 프레임 가시성, 그리고 영향을 미치는 객체의 가시성 판정 결과에 대해 총 8개의 조합이 존재하며, 정리하면 다음의 표와 같다.

	#n	#n+1	최종
(1)	가시	가시	가시
(2)	비가시	가시	가시
(3)	가시	비가시	가시
(4)	비가시	비가시	가시
(5)	가시	가시	비가시
(6)	비가시	가시	비가시
(7)	가시	비가시	비가시
(8)	비가시	비가시	비가시

[표 1] 가시성 판정 결과 조합

(1), (2), (7), (8)의 경우 올바르게 판정된 것이고, (3)의 경우는 시각적 결과에 영향을 미치지 않으므로 성능상 감내할 수 있다. (4)의 경우는 발생이 물리적으로 불가능하며, (6)의 경우는 한 프레임 늦게 가시성이 반영될 수 있으나 시각적 영향이 미미하다. 따라서 주의 깊게 봐야 할 것은 (5)의 경우이다. 해당 상황은 인스턴스 데이터의 재배치로 인해 발생할 수 있다. 예를 들어, 특정 인스턴스 i가 프레임 n에서 인덱스 p, 프레임 n+1에서 인덱스 q에 기록되며 $p \neq q$ 인 경우, CPU가 읽는 시점에 두 위치 모두에서 해당 인스턴스의 최신 가시성 정보가 반영되지 않을 수 있다. 이 경우, 실제로는 두 프레임 동안 보였던 인스턴스들이 누락되어 객체 전체가 보이지 않는 것으로 잘못 판정된다. 이러한 문제를 방지하기 위해, GPU가 갱신하는 가시성 플래그 버퍼를 두 개의 슬롯을 갖는 링 버퍼로 구성한다. 그리고 CPU측 메모리에 슬롯별 컬링 대상 인스턴스 수를 별도로 저장한다. 각 프레임 렌더링 시 서로 다른 슬롯에 번갈아가며 가시성 판정 결과를 기록하며, 이를 통해 적어도 한 프레임에 대해 모든 인스턴스의 가시성 정보가 완전하게 보존되도록 한다. CPU는 두 슬롯을 모두 읽으며 가시성을 판별하는데, 별도로 기록된 슬롯별 인스턴스 수 이상으로는 읽지 않도록 한다. 이때, 두 슬롯에서 인스턴스가 중복될 수 있지만 가시성 판정 단계에서만 존재하는 중복이므로 성능을 유의미하게 감소시키지는 않는다.



[그림 3] 인스턴스 누락 시나리오

애니메이션 우선순위

$$w_d = \frac{1}{1 + (d \times k_d)^2}$$
$$w_t = 1 + \frac{t}{t + k_t}$$
$$priority = w_d \times w_t$$

d : 카메라와 객체 사이의 거리, k_d : 거리 규모 상수, t : 마지막 갱신으로부터 경과한 시간, k_t : 시간 규모 상수, $priority$: 애니메이션 갱신 우선순위

가까운 객체와 오래 갱신되지 않은 객체의 갱신 우선순위를 높인다.

결론

본 연구에서는 GPU-CPU 동기화를 요구하지 않는 가시성 피드백 구조를 설계하여, 파이프라인 지연 없이 CPU로 가능한 한 최신의 가시성 정보를 활용할 수 있는 방법을 제시하였다. 이를 통해 추가 GPU 비용 없이 GPU에서 계산한 가시성 정보를 애니메이션 스케줄링에 반영하여 복잡한 애니메이션 연산을 효과적으로 줄일 수 있음을 보였다. 해당 방법은 다른 GPU-Driven 렌더링 파이프라인과 연계될 수 있으며, 하드웨어 인스턴싱 단위를 유지하며 수행될 수 있다. 최신 GPU-Driven 렌더링 파이프라인은 실시간으로 계산되는 작은 클러스터 단위로 LOD 적용 및 인스턴싱을 수행하는 경우가 많다. 보다 세밀한 단위의 가시성을 역전파하는 것은 새로운 연구 과제가 될 수 있다. 또한 본 연구는 GPU 컬링 결과를 애니메이션 갱신 최적화에 반영하는 것에 중점을 두었지만, 해당 결과를 물리 엔진, 게임의 전투 시스템 등 가시성을 효율적으로 활용할 수 있는 CPU 연산에 반영하는 것 역시 새로운 연구 과제가 될 수 있다.

참고문헌

- [1] B. Dudash, Skinned Instancing, NVIDIA SDK 10, 2007. [2] N. Greene et al., Hierarchical Z-buffer visibility, SIGGRAPH, 1993. [3] H. Zhang et al., Visibility Culling Using Hierarchical Occlusion Maps, SIGGRAPH, 1997. [4] U. Haar and S. Aaltonen, GPU-Driven Rendering Pipelines, SIGGRAPH Course, 2015. [5] B. Karis et al., Nanite, SIGGRAPH Course, 2021.